

Data Catalog MCP: Servidor MCP de Catálogo de Datos con Validación de Calidad

Redes de Computadoras - CC3067

José Antonio Mérida Castejón

6 de septiembre de 2026

1. Especificación del servidor MCP

1.1. Caso de uso

El servidor implementa un catálogo de datos interno con validación de calidad, dirigido al equipo de analítica de una empresa mediana. El problema que atiende es común en organizaciones que acumulan datasets de distintas fuentes (exportaciones de CRM, ventas, recursos humanos) sin documentación centralizada: los analistas invierten tiempo considerable en determinar qué datos existen, qué significa cada columna, en qué unidades está expresada, y si un extracto recibido cumple las condiciones mínimas de calidad antes de usarlo. El servidor expone herramientas de solo lectura y sin estado que el chatbot anfitrión coordina en lenguaje natural; el LLM coordina las llamadas e interpreta los resultados, pero no genera información por su cuenta.

El escenario concreto se enmarca como **Meridian Holdings**, una empresa ficticia con cuatro unidades de negocio (Meridian Mobile, Meridian Retail, Meridian Commerce y Recursos Humanos corporativos), cada una con su propio sistema de origen. El patrón corresponde a herramientas de uso industrial ya establecido — los archivos `schema.yml` de dbt, las suites de expectativas de Great Expectations, y catálogos de datos como DataHub o Amundsen — sin constituir un *feature store*, ya que no calcula ni sirve valores de *features* para inferencia.

1.2. Origen de la información

El servidor obtiene su información de dos fuentes complementarias, ambas locales al servidor y sin dependencias externas en tiempo de consulta:

1. **Archivos de datos en formato Parquet** (`data/*.parquet`), sobre los cuales se calculan estadísticas y validaciones. Cinco datasets: suscriptores de Meridian Mobile, ventas de Meridian Retail, transacciones de comercio electrónico de Meridian Commerce, rotación de personal de RR.HH., y un extracto mensual sin validar (copia de suscriptores con inconsistencias introducidas de forma controlada y reproducible, para demostrar `validate_dataset`).
2. **Catálogo declarativo** en un archivo YAML (`catalog.yaml`), escrito manualmente, que actúa simultáneamente como diccionario de datos y definición de reglas de calidad. Para cada

columna se declara su tipo, descripción, unidad y las reglas que debe cumplir:

```
- name: customerID
  type: string
  description: Unique customer identifier
  unit: null
  rules:
    - not_null
    - unique
    - regex: '^\\d{4}-[A-Z]{5}$'
      severity: error
```

Las reglas de calidad operan a nivel de columna (`not_null`, `unique`, `range`, `allowed_values`, `regex`, `type_check`) o a nivel de dataset (`row_count`, `freshness`). Cada regla declara una severidad: `error` invalida el dataset para uso posterior; `warning` señala una anomalía que amerita revisión sin bloquear el flujo de trabajo.

1.3. Arquitectura interna

El servidor (`cmd/server`) está escrito en Go, sin SDKs de MCP ni librerías de JSON-RPC de terceros — todo el framing de mensajes y el manejo del protocolo se implementó a mano, según lo exigido por el enunciado. Los paquetes relevantes:

- `internal/jsonrpc` — framing JSON-RPC 2.0 sobre un `io.Reader/io.Writer` genérico: lee mensajes delimitados por línea, los decodifica a la estructura `Message` (`jsonrpc`, `id`, `method`, `params`, `result`, `error`), y distingue solicitud de notificación según la presencia del campo `id`.
- `internal/mcp` — implementa el ciclo de vida MCP sobre esa capa: `initialize`, `notifications/initialized`, `tools/list` y `tools/call`, más el despacho a cada herramienta (un archivo `tool_*.go` por herramienta).
- `internal/catalog` — parseo de `catalog.yaml` a estructuras tipadas.
- `internal/data` — resuelve cada entrada del catálogo al archivo Parquet correspondiente y lo lee.
- `internal/embeddings` — cliente de embeddings con dos proveedores intercambiables (Ollama local, Gemini remoto) para `search_catalog`.
- `internal/config` — configuración vía variables de entorno (`TRANSPORT`, `CATALOG_PATH`, `DATADIR`, `PORT`, etc.).

Esta separación permite que la *misma* implementación se ejecute sin modificaciones tanto de forma local (stdio) como remota (HTTP/Cloud Run): el núcleo MCP es agnóstico al transporte.

1.4. Herramientas expuestas (endpoints MCP)

Todas las herramientas se invocan mediante el método JSON-RPC `tools/call`, identificadas por su campo `name` y con argumentos en `params.arguments`. La Tabla 1 detalla parámetros y tipo de retorno de cada una.

Herramienta	Parámetros y descripción
<code>ping</code>	Sin parámetros. Retorna " pong "; verificación de salud.
<code>list_datasets</code>	Sin parámetros. Retorna nombre, descripción, origen, número de filas y fecha de actualización de cada dataset registrado.
<code>describe_dataset</code>	<code>dataset</code> (string, requerido). Retorna el diccionario de datos completo: columnas, tipos, descripciones, unidades y reglas declaradas.
<code>profile_column</code>	<code>dataset</code> , <code>column</code> (string, requeridos). Retorna estadísticas según el tipo: cuantiles y atípicos para numéricas, cardinalidad y valores frecuentes para categóricas, rango y vacíos para fechas.
<code>validate_dataset</code>	<code>dataset</code> (string, requerido). Ejecuta las reglas declaradas contra los datos reales; retorna veredicto general (reglas evaluadas/aprobadas/fallidas), detalle por regla con severidad y porcentaje de violaciones, y hasta cinco valores de ejemplo por violación.
<code>find_undocumented_columns</code>	<code>dataset</code> (string, requerido). Compara las columnas presentes en el Parquet contra las declaradas en el catálogo, para detectar documentación desactualizada.
<code>search_catalog</code>	<code>query</code> (string, requerido). Búsqueda semántica (embeddings, similitud coseno) o por palabra clave si el servicio de embeddings no está disponible. Consulta dos índices independientes — datasets y columnas — y retorna ambos conjuntos por separado, ya que sus puntajes no son comparables entre sí.
<code>chart_column</code>	<code>dataset</code> , <code>column</code> (requeridos). Genera un histograma, gráfica de barras o serie de tiempo según el tipo de columna, y lo retorna como bloque de contenido tipo imagen del protocolo MCP.

Cuadro 1: Herramientas expuestas por el servidor MCP y sus parámetros.

1.5. Búsqueda semántica en dos niveles

`search_catalog` construye al iniciar el servidor dos índices vectoriales: uno de datasets (nombre, descripción, origen) para consultas de granularidad amplia, y otro de columnas (nombre y descripción de la columna junto con la descripción de su dataset, ya que el nombre de una columna por sí solo suele ser demasiado breve para una representación vectorial informativa). La recuperación es por similitud coseno mediante multiplicación matricial; dado el tamaño reducido del índice (≈ 100 vectores), una búsqueda exhaustiva es apropiada y no se justifica una base de datos vectorial dedicada.

1.6. Privacidad y servicios externos

La generación de embeddings se delega a un servicio externo (Ollama local o Gemini), lo que evita incorporar un modelo local al contenedor y mantiene una imagen ligera con arranques en frío reducidos. El servidor transmite a ese servicio **únicamente metadatos** — nombres de dataset, nombres de columna, descripciones y unidades, redactados manualmente en el catálogo — y las consultas del usuario, nunca registros ni valores de los archivos de datos. Esta separación es una decisión de diseño deliberada que hace viable el caso de uso en un entorno donde los datos suelen estar sujetos a restricciones contractuales o regulatorias.

1.7. Transporte y despliegue

- **stdio** (local): mensajes JSON delimitados por línea sobre stdin/stdout del proceso hijo, spawnado por el anfitrión (`cmd/host` o `cmd/client`) según `client.json`.
- **HTTP** (remoto): un único endpoint (`POST/`) que recibe un objeto JSON-RPC por solicitud y responde con el resultado correspondiente. Se activa con `TRANSPORT=http`; el puerto se toma de la variable `PORT` inyectada por la plataforma.

El servidor remoto se desplegó en **Google Cloud Run**, construido mediante Cloud Build directamente desde el repositorio de GitHub (sin necesidad de Docker local): una imagen `distroless` de dos etapas (compilación Go, luego binario estático más `catalog/` y `data/*.parquet`). Las credenciales de embeddings (`EMBEDDINGS_API_KEY`) se inyectan vía Secret Manager, nunca como variable de entorno en texto plano. Ambos transportes exponen exactamente los mismos métodos MCP: `initialize`, `tools/list` y `tools/call`, por lo que el anfitrión usa el servidor remoto de forma idéntica al local, solo cambiando la entrada `url` por `command` en `client.json`.

2. Análisis de la comunicación (Wireshark)

Se realizaron dos capturas complementarias contra el mismo servidor MCP (mismo binario, mismo código, mismos mensajes JSON-RPC), diferenciadas únicamente por el transporte:

- **Captura remota:** el anfitrión conectado al servidor desplegado en Google Cloud Run (`data-catalog-mcp-53268160127.northamerica-south1.run.app`) sobre HTTPS. Filtrada en Wireshark con `ipv6.addr==2600:1900:4243:200::`. Al ser HTTPS, el contenido de aplicación viaja cifrado; esta captura se usa para el análisis de las capas de enlace, red y transporte (§2.2), incluyendo el handshake TCP y TLS reales contra el servidor en la nube.
- **Captura local:** el mismo anfitrión conectado al mismo servidor ejecutándose localmente con `TRANSPORT=http` (sin TLS), filtrada con `tcp.port==8091`. Dado que el formato y contenido de los mensajes JSON-RPC es idéntico independientemente del transporte, esta captura se usa para inspeccionar y clasificar los mensajes JSON-RPC en texto plano (§2.1), lo cual no es posible directamente sobre la captura remota sin descifrar la sesión TLS.

2.1. Mensajes JSON-RPC capturados

Se identifican tres tipos de mensaje JSON-RPC en la especificación: **solicitud** (`request`, contiene `id` y `method`), **respuesta** (`response`, contiene `id` y `result` o `error`), y **notificación** (`notification`, sin `id`, no espera respuesta — usada por MCP para `notifications/initialized`). Estos mensajes de sincronización (`initialize` / `notifications/initialized`) son los que establecen el protocolo antes de que el anfitrión pueda invocar herramientas; el resto son solicitud/respuesta ordinarias de operación.

2.2. Análisis por capa

Capa de enlace. En la captura remota (Figura 1), Wireshark reporta un encabezado **Ethernet II** real: dirección MAC origen `84:01:12:d3:19:e9` (identificada por el fabricante como **KaonGroup**, la interfaz de red local de la máquina que capturó el tráfico) y dirección MAC destino `0e:62:e7:72:da:65`. Esta última *no* es la MAC del servidor de Cloud Run — por diseño, una dirección MAC nunca sobrevive más de un salto — sino la del siguiente dispositivo en la ruta (el gateway/router

#	Tipo	Método	Observaciones
1	Solicitud	<code>initialize</code>	Handshake inicial, negocia <code>protocolVersion</code> y <code>capabilities</code> .
2	Respuesta	—	Responde al <code>id</code> de (1) con las capacidades del servidor.
3	Notificación	<code>notifications/initialized</code>	Sin <code>id</code> ; confirma que el cliente completó el handshake.
4	Solicitud	<code>tools/list</code>	Pide el catálogo de herramientas disponibles.
5	Respuesta	—	Responde con el arreglo <code>tools</code> (Tabla 1).
6	Solicitud	<code>tools/call</code>	Ej. <code>list_datasets</code> , sin argumentos.
7	Respuesta	—	<code>content</code> con el resultado en texto.

Cuadro 2: Clasificación de los mensajes JSON-RPC observados en la captura.

local). Esta es la única capa donde se opera sobre un segmento físico compartido; a partir de la capa de red la comunicación ya es de extremo a extremo lógico.

Capa de red. El mismo frame (Figura 1) muestra **IPv6**: dirección origen `2800:98:1124:aa7::2` (la máquina cliente) y destino `2600:1900:4243:200::` (la IP pública de Cloud Run, dentro del bloque de Google Cloud). Wireshark resuelve esta última mediante su base de datos GeoIP como ubicada en Estados Unidos — una aproximación geográfica basada en el bloque de direcciones, no necesariamente la región física exacta (`northamerica-south1` en la consola de Cloud Run es la región lógica del servicio, no implica que el bloque IP esté geolocalizado con esa misma granularidad en bases de datos GeoIP de terceros). No se observó fragmentación IP: todos los segmentos grandes se manejan a nivel de TCP, no de IP.

Capa de transporte. Se observa el **three-way handshake** completo de TCP (Figura 2): SYN (puerto efímero 43952 del cliente → puerto 443 del servidor), SYN, ACK (el paquete mostrado en detalle, con `Flags: 0x012`), y el ACK final del cliente que completa la conexión. Sobre esa conexión TCP ya establecida corre inmediatamente el **handshake TLS 1.3** (Figura 3): **Client Hello** — que incluye el campo SNI (*Server Name Indication*, `data-catalog-mcp-53268160127.northamerica-south1.run.app`), visible en claro pese a ser TLS, ya que el SNI viaja sin cifrar para que el servidor sepa qué certificado presentar — seguido de **Server Hello**, **Change Cipher Spec**. A partir de ahí toda la comunicación queda cifrada bajo TLS 1.3: sin acceso a las claves de sesión (p.ej. vía `SSLKEYLOGFILE`), Wireshark solo puede mostrar **Application Data** cifrada (Figura 4), no su contenido. Esto confirma que la capa de transporte (TCP) entrega los datos de forma confiable y ordenada independientemente de que estén cifrados o no — el cifrado es una responsabilidad de la capa superior (TLS, entre transporte y aplicación), no de TCP en sí.

Capa de aplicación. Dentro de los registros **Application Data** de TLS, Wireshark identifica el protocolo interno como **HTTP** (campo `[Application Data Protocol: Hypertext Transfer Protocol]` en la Figura 4), aunque no puede decodificar su contenido sin las claves de sesión. Por diseño, el servidor MCP expone un único endpoint HTTP (`POST/, Content-Type:application/json`) cuyo cuerpo es, a su vez, un mensaje JSON-RPC — es decir, JSON-RPC actúa como un sub-protocolo de aplicación dentro de HTTP. En la captura local (sin TLS, mismo servidor) este mismo intercambio es perfectamente legible; la Tabla 2 y las figuras referenciadas en §2.1 muestran

el contenido real de esos mensajes `initialize`, `tools/list` y `tools/call`.

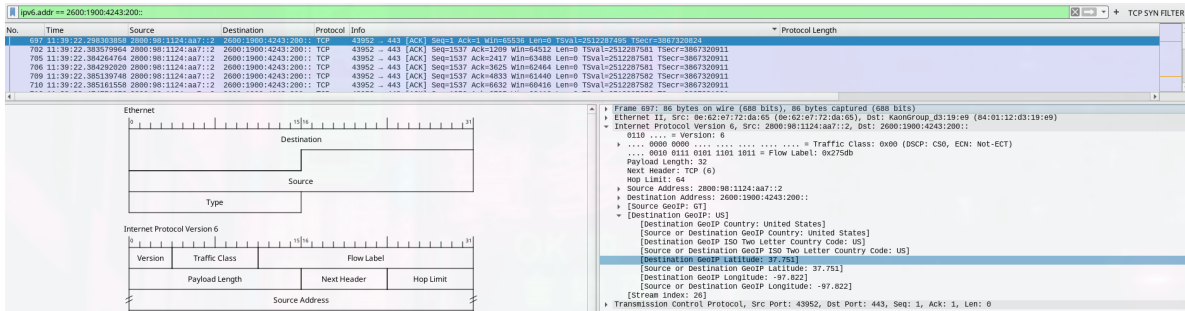


Figura 1: Captura remota: encabezado Ethernet II (capa de enlace) e IPv6 (capa de red) de un paquete ACK del handshake TCP contra Cloud Run.

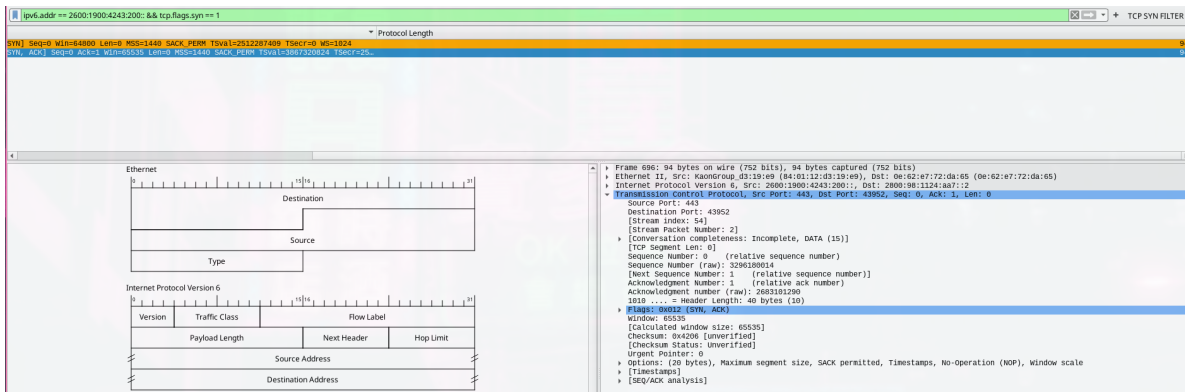


Figura 2: Captura remota: three-way handshake TCP (`tcp.flags.syn==1`), con el paquete SYN, ACK expandido mostrando puertos y flags.

3. Conclusiones y comentario sobre el proyecto

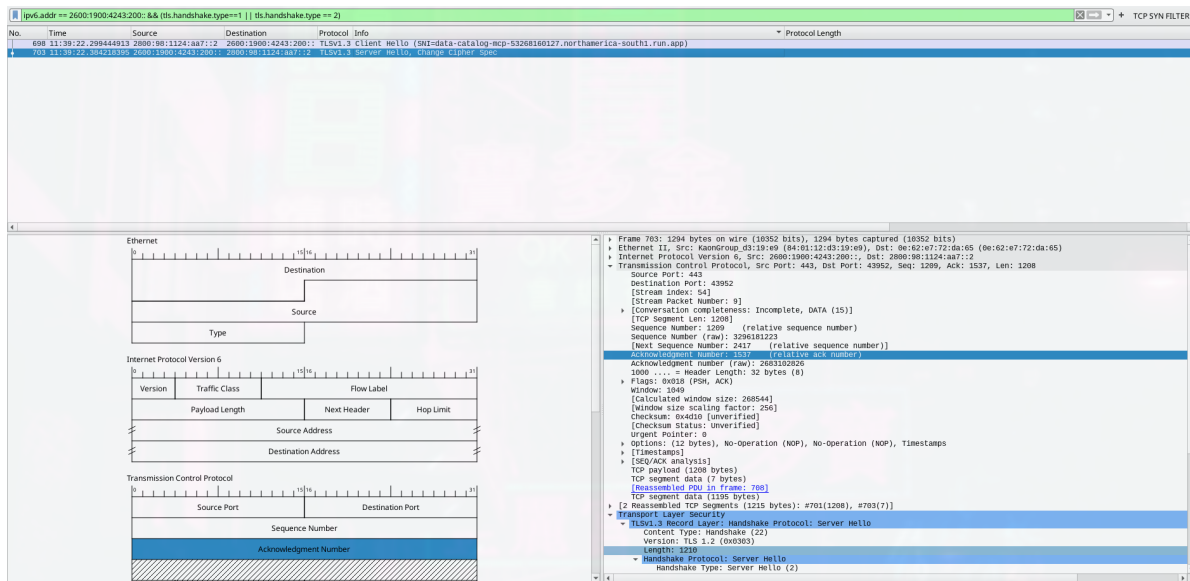


Figura 3: Captura remota: handshake TLS 1.3 (Client Hello con SNI, Server Hello, Change Cipher Spec).

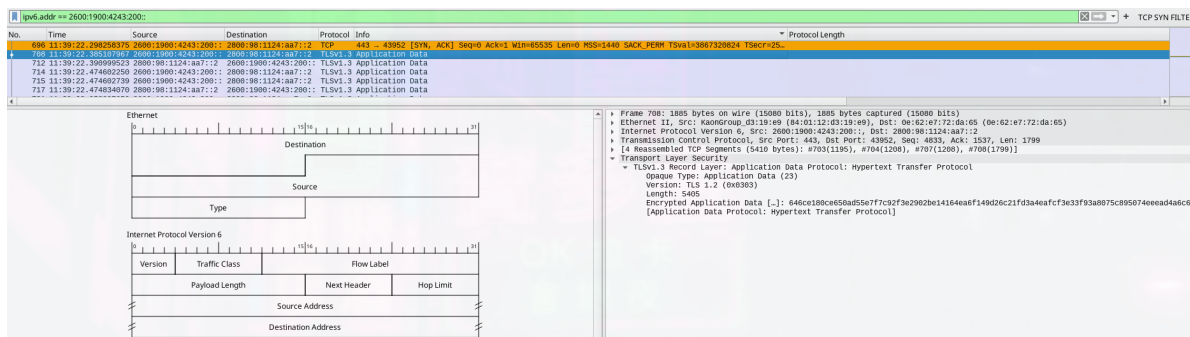


Figura 4: Captura remota: registro TLS de Application Data cifrado, identificado como HTTP a nivel de protocolo pero ilegible sin las claves de sesión.